

# Filo-Priori: A Multi-Edge Graph Attention Approach to Test Case Prioritization

Acauan C. Ribeiro\*, Eduardo L. Feitosa\*, Andre L. da Costa Carvalho\*,  
Eulanda M. dos Santos\*, Bruno F. Gadelha\*, Yan R. Soares\*, and Jose Nascimento<sup>†</sup> \*Instituto de  
Computação (IComp)

Universidade Federal do Amazonas (UFAM)

Manaus, AM, Brazil

{acauan.ribeiro, efeitosa, andre, emsantos, bruno, yan.soares}@icomp.ufam.edu.br <sup>†</sup>Motorola Mobility  
LLC

Manaus, AM, Brazil

josern@motorola.com

**Abstract**—In Continuous Integration (CI) environments, growing test suites make exhaustive testing impractical, motivating Test Case Prioritization (TCP) to maximize early fault detection. Existing TCP approaches, whether based on code coverage, historical failure rates, or deep learning, often treat test cases as independent entities. This assumption ignores the structural relationships that characterize real-world testing: tests that co-fail often share underlying code dependencies, semantically similar tests target related functionality, and temporal execution patterns reveal systematic connections.

We present FILO-PRIORI, a dual-stream deep learning approach that explicitly models these relationships through a Multi-Edge Test Case Relationship Graph encoding five complementary edge types: co-failure, co-success, component, semantic similarity, and temporal adjacency. The architecture combines a semantic stream processing Sentence-BERT embeddings of test descriptions and commit messages with a structural stream applying Graph Attention Networks (GATv2) over historical execution features on the relationship graph, unified through bidirectional cross-attention fusion.

We evaluate FILO-PRIORI against five state-of-the-art deep learning baselines (DeepOrder, TCP-Net, NodeRank, RETECS, and FailRank-BB) on two complementary datasets: an industrial CI/CD pipeline comprising 52,102 test executions and the RTPTorrent benchmark with 20 open-source Java projects. On the industrial dataset, FILO-PRIORI achieves an APFD of 0.7595, outperforming all baselines by +10.2% to +27.6% ( $p < 0.001$ ). On RTPTorrent, it achieves 0.8376. An ablation study reveals that the multi-edge graph is the most critical component, contributing 17.0% to performance, confirming that modeling structural relationships between test cases substantially improves fault detection over independent-test approaches.

**Index Terms**—Test Case Prioritization, Graph Attention Networks, Deep Learning, Continuous Integration, Dual-Stream Architecture, Class Imbalance



## 1 INTRODUCTION

CONTINUOUS Integration (CI) has become a fundamental practice in modern software development, enabling teams to integrate code changes frequently and detect defects early [1], [2]. A key challenge in CI environments is managing the growing test suite: as software evolves, the number of test cases increases, making it impractical to execute all tests for every commit [3].

Test Case Prioritization (TCP) addresses this challenge by ordering test cases to maximize early fault detection [4], [5]. The goal is to execute tests most likely to fail first, providing faster feedback to developers. The effectiveness of TCP is typically measured using the Average Percentage of Faults Detected (APFD) metric [6].

However, existing TCP approaches, whether based on coverage [4], historical failure [7], or machine learning [8], [9], often treat test cases as independent entities. This assumption ignores the rich relationships inherent in test suites: tests that co-fail in the same build often share underlying dependencies, tests with similar descriptions tar-

get related functionality, and historical execution patterns frequently reveal systematic structural connections [10].

To leverage these relationships, we propose combining two complementary information sources for TCP: (1) *semantic information* derived from test descriptions and commit messages, and (2) *structural information* extracted from execution histories and test relationships. This dual-stream approach captures both what tests intend to verify (semantics) and how they historically behave (structure). Our key insight is that neither modality is sufficient in isolation. Semantic similarity alone may mislead, as tests with similar descriptions can exhibit divergent failure patterns when targeting distinct code paths. Conversely, tests that frequently co-fail may have entirely dissimilar descriptions while sharing latent code dependencies.

To address these limitations and operationalize this insight, we present FILO-PRIORI, a novel deep learning approach for TCP that synergizes semantic and structural information through four key innovations:

- 1) **Multi-Edge Test Case Relationship Graph:** We

build a graph that captures multiple types of relationships between test cases: co-failure edges (tests that fail together), co-success edges (tests that pass together), and semantic similarity edges (tests with similar descriptions). This multi-edge approach increases graph density and captures complementary relationships.

- 2) **Dual-Stream Architecture:** We employ a dual-stream model that processes semantic features (embeddings of test descriptions and commit messages) and structural features (historical execution patterns) through separate neural networks before fusion. The dual-stream design is motivated by the observation that semantic content and behavioral patterns are complementary signals that require different processing strategies: text embeddings benefit from feed-forward refinement while structural features benefit from graph-based message passing.
- 3) **Graph Attention Networks for Structural Learning:** We use Graph Attention Networks (GAT) [11] to learn representations that capture test relationships, allowing the model to propagate information between related test cases with learned attention weights.
- 4) **Weighted Focal Loss for Class Imbalance:** We employ Weighted Focal Loss [12] to address the severe class imbalance (37:1 Pass:Fail ratio) inherent in test execution data, focusing training on hard-to-classify examples while down-weighting easy negatives.

We evaluate FILO-PRIORI on two complementary datasets: (1) an industrial CI/CD pipeline for mobile devices with 52,102 test executions across 1,339 builds, where a build represents a complete cycle of code integration and automated testing, and (2) the RTPTorrent benchmark with 20 open-source Java projects containing over 100,000 Travis CI builds. Our evaluation addresses four research questions:

- **RQ1:** Can modeling structural relationships between test cases through graph neural networks improve fault detection compared to approaches that treat tests independently?
- **RQ2:** Which architectural components contribute most to the effectiveness of graph-based test case prioritization?
- **RQ3:** How robust is the proposed approach when applied to builds from different time periods than those used for training?
- **RQ4:** How sensitive is the approach to the choice of key hyperparameters?

Our results show that FILO-PRIORI achieves an APFD of 0.7595 on the industrial dataset, outperforming all five deep learning baselines by +10.2% to +27.6% ( $p < 0.001$ ), and 0.8376 on RTPTorrent across 20 open-source Java projects. The ablation study reveals that the multi-edge graph construction is the most critical component (17.0% contribution), validating our central hypothesis that modeling structural relationships between test cases substantially improves fault detection.

Therefore, this paper makes the following contributions:

- 1) **Architectural:** We propose a dual-stream neural architecture that combines Sentence-BERT embed-

dings with Graph Attention Networks (GATv2) operating over a multi-edge test case relationship graph that encodes five complementary edge types (co-failure, co-success, component, semantic similarity, and temporal adjacency), unified through bidirectional cross-attention fusion.

- 2) **Feature Engineering:** We identify 19 discriminative structural features from an initial set of 29, selected through importance analysis and correlation filtering, combining base historical features with DeepOrder-inspired temporal features.
- 3) **Empirical:** We conduct a comprehensive evaluation against five state-of-the-art deep learning baselines (DeepOrder, TCP-Net, NodeRank, RETECS, and FailRank-BB) on two complementary datasets spanning industrial and open-source domains, demonstrating consistent state-of-the-art performance.
- 4) **Practical:** We provide a complete replication package including source code, trained models, and configuration files, enabling reproducibility and extension of our approach.

**Paper Organization.** The remainder of this paper is organized as follows. Section 2 presents the fundamental background concepts that underlie our approach, including TCP, the APFD metric, Graph Attention Networks, and class imbalance handling. Section 3 discusses related work on TCP, reviewing traditional, machine learning-based, and graph-based approaches, and positioning FILO-PRIORI relative to existing methods. Section 4 describes the FILO-PRIORI approach in detail, including its dual-stream architecture, feature extraction, graph construction, and training strategy. Section 5 presents the experimental design, including datasets, baselines, evaluation metrics, and implementation details. Section 6 reports the experimental results organized by research questions. Section 7 discusses the implications of our findings and the practical considerations for adopting FILO-PRIORI. Section 8 addresses threats to the validity of our study. Finally, Section 9 concludes the paper and outlines directions for future work.

## 2 BACKGROUND

This section introduces fundamental concepts underlying our approach.

### 2.1 Test Case Prioritization

Test Case Prioritization (TCP) is the process of ordering test cases for execution to achieve certain objectives, such as maximizing early fault detection [4]. Formally, given a test suite  $T$  and a permutation function  $PT$  that produces ordered sequences of  $T$ , TCP aims to find an ordering  $T' \in PT$  that maximizes a given objective function [5].

In Continuous Integration environments, TCP is particularly important because: (1) test suites grow over time, making exhaustive testing impractical [3]; (2) developers need rapid feedback on code changes [1]; and (3) computing resources for testing are often limited [8].

To illustrate the impact of effective prioritization, consider a test suite with 100 test cases where only 3 detect faults. Under random ordering, developers may need to

execute 50 or more tests before encountering a failure. An effective prioritization strategy would place these 3 failing tests near the beginning of the execution order, providing faster feedback and enabling developers to begin debugging sooner.

## 2.2 APFD Metric

To quantify the effectiveness of a prioritization strategy, the research community has adopted standardized metrics. The most widely used is the Average Percentage of Faults Detected (APFD) [6]. For a test suite  $T$  containing  $n$  test cases that detect  $m$  faults, with  $TF_i$  being the position of the first test case that detects fault  $i$ , APFD is calculated as:

$$\text{APFD} = 1 - \frac{\sum_{i=1}^m TF_i}{n \times m} + \frac{1}{2n} \quad (1)$$

It ranges from 0 to 1, where higher values indicate better prioritization. An APFD of 0.5 corresponds to random ordering, while 1.0 indicates prioritization where all faults are detected by the first tests.

## 2.3 Graph Neural Networks

Graph Neural Networks (GNNs) are a class of deep learning models designed to operate on graph-structured data [13]. The core principle underlying GNNs is *message passing*: each node updates its representation by aggregating information from its neighbors, enabling the model to capture structural patterns in the graph topology. Formally, a GNN layer computes:

$$h_i^{(l+1)} = \text{UPDATE} \left( h_i^{(l)}, \text{AGGREGATE} \left( \{h_j^{(l)} : j \in \mathcal{N}(i)\} \right) \right) \quad (2)$$

where  $h_i^{(l)}$  is the representation of node  $i$  at layer  $l$ ,  $\mathcal{N}(i)$  is the set of neighbors of node  $i$ , and UPDATE and AGGREGATE are learnable functions.

Several GNN architectures differ in how they implement these functions: Graph Convolutional Networks (GCN) [14] use normalized adjacency for aggregation, GraphSAGE [15] samples and aggregates from local neighborhoods, and Graph Attention Networks (GAT) [11] introduce learned attention mechanisms to weigh neighbor contributions. In this work, we adopt GAT due to its ability to learn the relative importance of different neighbors, which is particularly relevant for test case prioritization where different types of test relationships (e.g., co-failure vs. semantic similarity) carry different predictive value.

## 2.4 Graph Attention Networks

Graph Attention Networks (GAT) [11] are a specific GNN architecture that incorporates attention mechanisms to weigh the importance of neighboring nodes. For a node  $i$  with neighbors  $\mathcal{N}_i$ , GAT computes:

$$\vec{h}_i' = \sigma \left( \sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W} \vec{h}_j \right) \quad (3)$$

where  $\alpha_{ij}$  are attention coefficients computed as:

$$\alpha_{ij} = \text{softmax}_j \left( \text{LeakyReLU} \left( \vec{a}^T [\mathbf{W} \vec{h}_i \parallel \mathbf{W} \vec{h}_j] \right) \right) \quad (4)$$

Brody et al. [16] showed that standard GAT computes a restricted form of “static” attention where the ranking of attention scores is independent of the query node. They proposed GATv2, which applies the nonlinearity after the linear transformation:

$$\alpha_{ij} = \text{softmax}_j \left( \vec{a}^T \cdot \text{LeakyReLU} \left( \mathbf{W} [\vec{h}_i \parallel \vec{h}_j] \right) \right) \quad (5)$$

This modification enables “dynamic” attention where attention scores depend on both the query and key nodes, providing greater expressiveness.

## 2.5 Class Imbalance and Focal Loss

Test execution data typically exhibits severe class imbalance, with far more passing tests than failing tests [8], [17]. Standard cross-entropy loss is dominated by the majority class, leading to models that predict “pass” for nearly all tests.

**Focal Loss** [12] addresses this by down-weighting easy examples and focusing on hard ones:

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \quad (6)$$

where  $p_t$  is the predicted probability of the true class,  $\alpha_t$  is a class-balancing weight, and  $\gamma$  is the focusing parameter. When  $\gamma > 0$ , the  $(1 - p_t)^\gamma$  term reduces the loss for well-classified examples, focusing training on hard negatives.

## 2.6 Cross-Attention Mechanisms

Cross-attention allows one sequence (or modality) to attend to another, enabling information exchange between different representations. Given queries  $Q$  from one modality and keys/values  $K, V$  from another:

$$\text{CrossAttn}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (7)$$

In multi-modal fusion, bidirectional cross-attention allows each modality to selectively attend to relevant parts of the other, combining information in a learned, adaptive manner.

## 3 RELATED WORK

This section reviews prior work on TCP. We organize related work into three groups reflecting the evolution of TCP methods: (1) traditional and coverage-based approaches, which rely on code structure and execution information; (2) machine learning and deep learning approaches, which learn prioritization patterns from historical data; and (3) graph-based approaches, which explicitly model relationships between test artifacts. Each group represents a different strategy for capturing the information needed for effective prioritization.

### 3.1 Traditional and Coverage-based TCP

Early TCP research focused on code coverage-based techniques. Rothermel et al. [4] proposed total and additional coverage prioritization strategies, where test cases are ordered based on the number of code statements or branches they cover. Li et al. [18] demonstrated the effectiveness of search-based approaches for TCP, exploring genetic algorithms for optimizing test orderings. Walcott et al. [19] proposed time-aware TCP techniques that consider both coverage and execution cost. The advantage of coverage-based approaches is their strong theoretical foundation and direct connection to code structure; however, they require instrumentation and access to source code, which limits their applicability in black-box testing scenarios. Elbaum et al. [5] conducted extensive empirical studies comparing coverage-based strategies, showing that the effectiveness of different techniques varies significantly across programs and versions, and that no single strategy dominates in all scenarios. Henard et al. [20] compared white-box and black-box TCP techniques at scale, finding that similarity-based approaches can be competitive with coverage-based methods while requiring less instrumentation. History-based approaches leverage past test execution results. Kim and Porter [7] showed that tests that failed recently are more likely to fail again, a simple yet effective heuristic. The main advantage of history-based methods is their low computational cost and independence from source code; their limitation is the inability to handle new test cases or tests that have not yet failed.

### 3.2 Machine Learning and Deep Learning for TCP

Machine learning has been increasingly applied to TCP, aiming to learn complex patterns from historical execution data. Spieker et al. [8] introduced RETECS (Reinforcement learning for automatic Test Case prioritization and Selection in Continuous Integration), a pioneering approach that frames TCP as a reinforcement learning problem. RETECS treats each CI cycle as an episode and learns to prioritize tests by maximizing a reward signal based on fault detection. Its advantage is the ability to continuously adapt to changing test patterns without retraining; however, it requires a significant number of CI cycles to converge and may struggle with sparse failure signals.

Chen et al. [17] proposed DeepOrder, a deep learning approach for TCP in continuous integration testing. DeepOrder uses a deep neural network that takes as input eight historical features per test case (including last execution result, time since last failure, and failure rate) to predict the likelihood of failure. The predicted probabilities are then used to rank test cases. DeepOrder demonstrated improvements over simple heuristics; however, it treats test cases independently without considering relationships between them, and it relies solely on historical features without incorporating semantic information from test descriptions or commit messages.

TCP-Net [21] introduced an end-to-end deep learning approach that learns directly from test execution data, aiming to capture temporal patterns in test execution sequences. While it showed promising results, it also treats tests as

independent entities and does not model structural relationships.

Busjaeger and Xie [22] proposed a learning-to-rank approach for TCP that combines multiple information sources (code coverage, text similarity, and historical data) into a unified ranking model, demonstrating the value of multi-source feature integration. Bagherzadeh et al. [23] conducted a comprehensive study of reinforcement learning approaches for TCP, finding that while RL methods can adapt to changing environments, they often require substantial training data to converge. Bertolino et al. [24] explored learning-based TCP in industrial CI settings, identifying challenges such as test suite evolution and flaky tests that affect practical deployment. Pan et al. [9] proposed a test case prioritization approach that combines multiple information sources using attention mechanisms, demonstrating improvements over single-feature approaches.

Hernandes et al. [25] proposed a method for ordering regression testing plans in the context of non-automated, black-box testing. Their approach uses Sentence-BERT to extract embeddings from test case scripts and commit messages, then employs machine learning classifiers (KNN, Logistic Regression, MLP, XGBoost, Random Forest) to predict the failure probability of each test case. Test cases are then ordered by their predicted failure probability. Their experiments showed results equal to or better than human experts in 92.52% to 94.24% of scenarios. While this work demonstrated the value of semantic information from test descriptions and commits for TCP, it does not model structural relationships between test cases and relies on traditional machine learning classifiers rather than graph-based architectures. FILO-PRIORI builds upon this work by incorporating structural features and graph-based learning to capture test relationships.

### 3.3 Graph-Based Approaches for Testing

Li et al. [26] proposed NodeRank, a test input prioritization approach for GNNs using mutation-based analysis. NodeRank leverages ensemble learning-based ranking models (including LightGBM and Random Forest) with three types of mutations: graph structure mutations (edge additions/removals), node feature mutations (Gaussian noise injection), and model mutations (weight perturbation). For each test input (graph node), NodeRank computes a priority score based on how sensitively the model's prediction changes under these mutations. NodeRank was evaluated on Cora, CiteSeer, and PubMed citation network datasets, outperforming uncertainty-based baselines (DeepGini, entropy-based methods) by significant margins. While NodeRank targets GNN model testing rather than software TCP, it demonstrates the potential of graph-based and ensemble learning approaches for test prioritization. A key limitation is its reliance on mutation-based analysis, which requires running the model multiple times per test input, increasing computational cost.

### 3.4 Summary and Positioning of FILO-PRIORI

The reviewed literature reveals several gaps that FILO-PRIORI aims to address. Traditional coverage-based approaches require source code access and cannot capture

TABLE 1: Positioning of FILO-PRIORI against deep learning baselines.

| Method             | Semantic | Structural | Graph | Cross-Attn | Orphan |
|--------------------|----------|------------|-------|------------|--------|
| DeepOrder          | –        | ✓          | –     | –          | –      |
| TCP-Net            | –        | ✓          | –     | –          | –      |
| NodeRank           | –        | ✓          | ✓     | –          | –      |
| RETECS             | –        | ✓          | –     | –          | –      |
| FailRank-BB        | ✓        | –          | –     | –          | –      |
| <b>FILO-PRIORI</b> | ✓        | ✓          | ✓     | ✓          | ✓      |

complex inter-test relationships. Deep learning approaches (RETECS, DeepOrder, TCP-Net) have demonstrated the value of learning from historical data, but they treat test cases independently, missing structural dependencies. FailRank-BB [25] demonstrated the value of semantic information from test descriptions and commits, but relies on traditional classifiers (Logistic Regression) without modeling inter-test relationships. Graph-based approaches (NodeRank) have shown the potential of modeling relationships, but focus on GNN model testing rather than software TCP.

Table 1 summarizes the key differences between FILO-PRIORI and the five deep learning baselines used in our evaluation.

FILO-PRIORI distinguishes itself from these approaches by: (1) explicitly modeling test case relationships through a multi-edge graph encoding five complementary edge types (co-failure, co-success, component, semantic similarity, and temporal adjacency); (2) combining both semantic and structural information in a dual-stream architecture, extending the semantic approach of Hernandez et al. [25] with graph-based structural learning; and (3) using GATv2 [16] to propagate failure signals between related test cases with dynamic attention, enabling the model to capture multi-hop dependencies that simpler approaches miss. We evaluate FILO-PRIORI on two complementary datasets spanning industrial TCP and open-source projects to demonstrate generalization across domains.

## 4 APPROACH: FILO-PRIORI

This section describes the FILO-PRIORI approach for Test Case Prioritization.

### 4.1 Overview

The FILO-PRIORI architecture is a dual-stream system that takes as input: (1) test case descriptions and, when available, commit messages (semantic); (2) historical test execution patterns (structural); and (3) a test relationship graph. It outputs a ranking of test cases by predicted failure probability. Figure 1 presents an overview of the framework.

The approach consists of three main modules:

- 1) **Semantic Stream:** A feed-forward network that processes text embeddings of test descriptions and commit messages, capturing textual similarity between test cases and code changes.
- 2) **Structural Stream:** A Graph Attention Network (GAT) that processes historical features over the test relationship graph, learning to propagate failure signals between related tests.

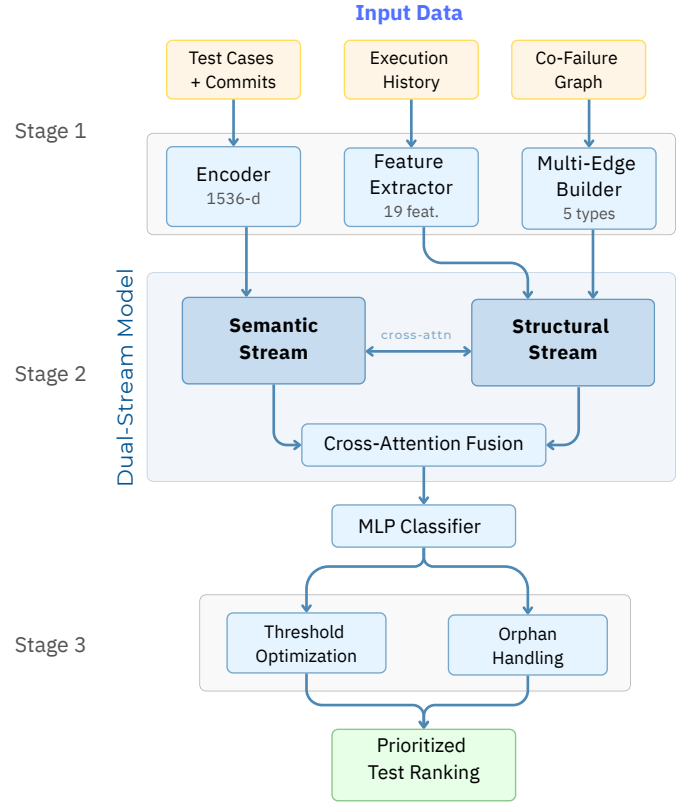


Fig. 1: Overview of the FILO-PRIORI framework. The system processes test case descriptions and execution history through dual semantic and structural streams, fuses representations via cross-attention, and applies post-processing for orphan handling and threshold optimization.

- 3) **Cross-Attention Fusion:** Bidirectional cross-attention that fuses semantic and structural representations, allowing each modality to attend to the other for final classification.

This dual-stream design captures both what tests do (semantics) and how they behave (structure), combining complementary information sources. Before detailing each of these modules, however, we describe the process of extracting semantic and structural features, as well as the test relationship graph generation.

### 4.2 Semantic Feature Extraction

In this paper, we use Sentence-BERT [27] with the `all-mpnet-base-v2` model to encode textual information. We chose Sentence-BERT because it produces sentence-level embeddings that capture semantic similarity between texts, which is essential for comparing test case descriptions and commit messages. Unlike word-level embeddings, Sentence-BERT generates fixed-size vectors that represent the overall meaning of a sentence, enabling efficient similarity computation between test artifacts.

For each test case, we concatenate two textual fields:

- **Test case summary (TC\_Summary):** A brief description of what the test case verifies (e.g., “Verify that OTA download upgrade package installs correctly after network reconnection”).

- **Test case steps** (TC\_Steps): The detailed sequence of actions to be performed during test execution (e.g., “1. Connect device to Wi-Fi. 2. Navigate to Settings & System Update. 3. Initiate download...”).

When available, we also encode information about the associated code changes:

- **Commit message**: A natural language description of the code modification made by the developer (e.g., “Fix null pointer exception in OTA download handler”).
- **Code diff**: The actual code changes, truncated to 2,000 characters to maintain a manageable input size while preserving the most relevant modifications.

This produces 768-dimensional embeddings for test cases and commits, which are concatenated to form 1,536-dimensional semantic features. It is worth noting that the semantic stream is designed to work with any sentence embedding model; in this work, we use Sentence-BERT, but alternative models could be employed depending on the domain and available data.

### 4.3 Structural Feature Extraction

We investigated a total of 29 candidate structural features capturing historical execution patterns. These features were identified through a combination of domain knowledge from the TCP literature [7], [17] and analysis of the available execution data. After feature selection (described below), 19 features were retained, organized into two categories.

**Base Features (10 features)**: These features capture the fundamental historical behavior of each test case:

- **test\_age**: Number of builds since the test case first appeared in the execution history, indicating its maturity.
- **failure\_rate**: The ratio of failed executions to total executions across the entire history of the test case.
- **recent\_failure\_rate**: Failure rate computed over the last 5 builds. We chose a window of 5 builds as it balances recency with statistical stability, following common practice in TCP literature [17].
- **flakiness\_rate**: The frequency of Pass↔Fail oscillations, computed as the number of status transitions divided by the total number of executions. High flakiness indicates unreliable tests.
- **consecutive\_failures**: The length of the current consecutive failure streak, capturing ongoing instability.
- **max\_consecutive\_failures**: The maximum consecutive failure streak observed in the test’s history, indicating its worst-case behavior.
- **failure\_trend**: The difference between recent failure rate and overall failure rate, where positive values indicate deteriorating performance.
- **commit\_count**: Number of unique commits associated with the test case’s execution history, reflecting code change activity.
- **cr\_count**: Number of distinct change requests (CRs) associated with the test, indicating the breadth of modifications.

TABLE 2: Multi-Edge Graph Edge Types

| Edge Type  | Weight | Formula                                   |
|------------|--------|-------------------------------------------|
| Co-Failure | 1.0    | $\min(P(t_j t_i), P(t_i t_j))$            |
| Co-Success | 0.5    | $\min(P(t_j t_i), P(t_i t_j))$            |
| Component  | 0.4    | $ C_i \cap C_j  /  C_i \cup C_j $         |
| Semantic   | 0.3    | $\cos(\mathbf{e}_i, \mathbf{e}_j)$        |
| Temporal   | 0.2    | $\text{count}_{adj} / \max(\text{count})$ |

- **test\_novelty**: A binary indicator (1 if the test case appears for the first time in the current build, 0 otherwise).

**DeepOrder-Inspired Features (9 features)**: Following the temporal feature engineering approach of DeepOrder [17], we added features that capture recent execution dynamics. The **execution\_status\_last\_N** features (for  $N \in \{1, 2, 3, 5, 10\}$ ) represent the proportion of failures in the last  $N$  executions, providing multi-scale windows that capture both immediate ( $N=1$ ) and short-term ( $N=10$ ) failure patterns. Additionally, we include **cycles\_since\_last\_fail** (number of builds since the last failure, normalized to  $[0, 1]$ ), **distance** (temporal distance from the last failure in calendar time, normalized), **status\_changes** (total number of Pass↔Fail transitions, capturing behavioral instability), and **fail\_rate\_last\_10** (failure rate over the last 10 executions, complementing the 5-build window).

These temporal features capture patterns that static metrics miss: a test that failed in the last execution is more likely to fail again than one that failed only in distant history.

**Feature Selection**. We initially extracted 29 candidate features. To reduce noise and improve generalization, we performed feature selection using a two-step process: (1) importance analysis using a preliminary Random Forest classifier to rank features by their contribution to failure prediction, and (2) correlation filtering to remove redundant features with Pearson correlation above 0.90. This process reduced the feature set from 29 to the 19 features described above, retaining only those with meaningful discriminative power.

### 4.4 Multi-Edge Test Relationship Graph

We construct a dense multi-edge graph where nodes represent test cases and edges capture complementary relationship types between them. Dense graph connectivity is critical for effective GAT message passing, since sparse graphs limit information propagation between related test cases. We define five edge types with empirically-tuned weights, as summarized in Table 2. The choice of five edge types reflects the different dimensions along which test cases can be related: behavioral (co-failure, co-success), textual (semantic similarity), temporal (execution adjacency), and organizational (component membership). Figure 2 illustrates the resulting graph structure.

We now describe each edge type in detail:

**Co-Failure Edges** (weight 1.0) connect tests that fail together in the same build, where the edge weight is the minimum conditional probability: if tests  $t_i$  and  $t_j$  co-failed  $n$  times, with  $t_i$  failing  $f_i$  times total, the weight is  $\min(n/f_i, n/f_j)$ . This symmetric formulation ensures robust correlation measurement. Co-failure edges receive the

Multi-Edge Test Relationship Graph

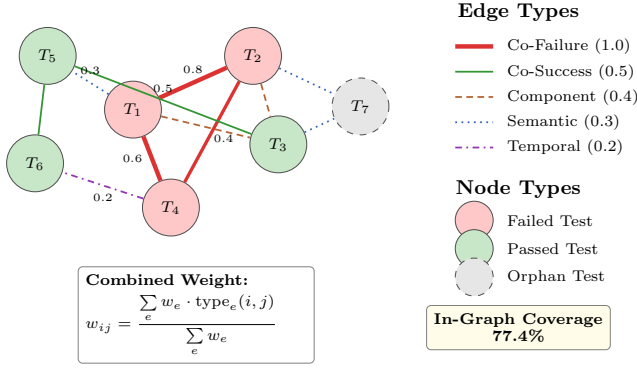


Fig. 2: Multi-edge test relationship graph with five edge types. Edge thickness and style indicate relationship type and strength. Failed tests (red) propagate failure signals through co-failure edges (weight 1.0), while orphan tests (dashed) connect through semantic edges (weight 0.3).

highest weight because they represent the strongest signal for related test behavior.

**Co-Success Edges** (weight 0.5) connect tests that pass together in the same build. The weight computation mirrors co-failure edges using co-pass conditional probabilities. While weaker than co-failure, co-success edges capture complementary information about test groupings, as tests that consistently pass together may share underlying dependencies.

**Component Edges** (weight 0.4) connect tests that belong to the same software component or module, computed as the Jaccard similarity of their component sets:  $|C_i \cap C_j| / |C_i \cup C_j|$ , where  $C_i$  denotes the set of components associated with test  $t_i$ . These edges capture organizational relationships that reflect architectural dependencies.

**Semantic Edges** (weight 0.3) connect tests with embedding similarity above a threshold  $\tau = 0.65$ , with  $k = 10$  nearest neighbors per test. This relaxed threshold (vs.  $\tau = 0.75$  in Hernandez et al. [25]) increases connectivity for previously isolated nodes, enabling the GAT to propagate information to tests with few behavioral edges.

**Temporal Edges** (weight 0.2) connect tests that are frequently executed adjacently (one after the other) within the same build. The edge weight is the normalized adjacency count:  $\text{count}_{adj} / \max(\text{count})$ . These edges capture scheduling patterns that may reflect implicit dependencies between tests.

For each test pair, we compute a combined edge weight as the weighted sum normalized by total weight:

$$w_{ij} = \frac{\sum_{e \in E} w_e \cdot \text{type}_e(i, j)}{\sum_{e \in E} w_e} \quad (8)$$

This multi-edge approach increases graph density from 0.02% (co-failure only) to 0.5–1.0% (all types), with 77.4% of test cases connected to the graph (vs. 50–60% with sparser configurations). This type of multi-relational graph construction is inspired by heterogeneous graph neural networks [13], adapted here to capture the multiple dimensions of test case relationships.

## 4.5 Semantic Stream

The Semantic Stream processes text embeddings through a feed-forward network with residual connections. The input consists of the concatenated embeddings of the test case description and commit message. In this work, we use Sentence-BERT embeddings, resulting in 1,536-dimensional input (768 dimensions for the test case + 768 for the commit). However, the architecture is agnostic to the specific embedding model; any sentence embedding model producing fixed-size vectors can be used.

The stream applies an input projection layer (from the embedding dimension to 256) followed by two residual feed-forward blocks. Residual connections [28] are employed to facilitate gradient flow and enable the network to learn incremental refinements over the projected embeddings, which is particularly important given the high dimensionality of the input. Each block consists of a linear expansion ( $256 \rightarrow 1,024$ ), GELU activation, dropout (0.3), linear compression ( $1,024 \rightarrow 256$ ), dropout (0.3), and layer normalization. These hyperparameters were selected through grid search on the validation set, balancing model capacity with regularization to prevent overfitting.

The output is a 256-dimensional semantic representation for each test case.

## 4.6 Structural Stream (Graph Attention Network)

The Structural Stream processes the selected structural features over the test relationship graph using Graph Attention Networks [11]. The input consists of the 19 structural features (10 base + 9 DeepOrder-inspired) per test case described in Section 4. The GAT architecture propagates information between related test cases through the multi-edge graph, allowing the model to learn from the failure patterns of neighboring tests.

We use two GAT layers with the following configuration. The first layer uses multi-head attention with 4 heads and concatenation:

$$h_i^{(1)} = \parallel_{k=1}^4 \sigma \left( \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k \mathbf{W}^k h_j^{(0)} \right) \quad (9)$$

where  $\alpha_{ij}^k$  are learned attention coefficients for head  $k$ . Each head produces a 256-dimensional output, which after concatenation yields a 1,024-dimensional intermediate representation. The second layer uses single-head attention with averaging, projecting back to 256 dimensions per test case. The choice of 4 heads for the first layer and 1 head for the second follows the common GAT practice [11] of using multi-head concatenation for intermediate layers and averaging for the final layer.

The GAT layers incorporate the combined edge weights from the multi-edge graph (Equation 7), allowing the model to learn the relative importance of different relationship types during attention computation. This means that co-failure connections (weight 1.0) naturally receive stronger attention than semantic similarity connections (weight 0.3), reflecting their stronger predictive signal for failure detection.



#### 4.7 Cross-Attention Fusion

The Cross-Attention Fusion module combines semantic and structural representations through bidirectional attention, allowing each modality to attend to the other, so that semantic features are informed by structural patterns and vice versa. This bidirectional exchange enables the model to discover correlations between textual content and behavioral patterns that neither stream could capture independently.

Specifically, the fusion operates in two directions. In the semantic-to-structural direction, semantic features use the structural representations as context:

$$h'_{sem} = h_{sem} + \text{MHA}(h_{sem}, h_{struct}, h_{struct}) \quad (10)$$

In the structural-to-semantic direction, structural features attend to the semantic representations:

$$h'_{struct} = h_{struct} + \text{MHA}(h_{struct}, h_{sem}, h_{sem}) \quad (11)$$

where  $\text{MHA}(Q, K, V)$  denotes multi-head attention [29] with query  $Q$ , key  $K$ , and value  $V$ . Each attention operation uses 4 heads with 256-dimensional queries, keys, and values, followed by layer normalization for training stability. The residual connections in Equations 9 and 10 ensure that the original representations are preserved while being enriched with cross-modal information.

The attended features from both directions are then concatenated to form the fused representation:

$$h_{fused} = [h'_{sem} || h'_{struct}] \quad (12)$$

The resulting 512-dimensional fused vector is passed to a classifier MLP consisting of two hidden layers with dimensions [128, 64], each followed by ReLU activation and dropout (0.4). The final output layer produces the failure probability for each test case.

Figure 3 illustrates the complete dual-stream architecture with the cross-attention fusion mechanism.

#### 4.8 Training with Simplified Dual-Balancing

Test execution data typically exhibits severe class imbalance, with far more passing tests than failing tests. In our industrial dataset, the Pass:Fail ratio is 37:1. Addressing this imbalance requires careful design, as multiple compensation mechanisms applied simultaneously can cause over-correction. Through preliminary experiments on the validation set, we observed that combining class weights, balanced sampling, and focal alpha together led to mode collapse where the model predicted all samples as the minority class.

Based on this empirical finding, we adopted a simplified dual-balancing strategy: the sampling handles the prior distribution while the focal loss stabilizes the discriminative bias. The loss function is a Focal Loss [12]:

$$\mathcal{L} = -\alpha \cdot (1 - p_t)^\gamma \cdot \log(p_t) \quad (13)$$

where  $\alpha = 0.5$  (neutral, no class preference) and  $\gamma = 2.0$  (the standard value proposed by Lin et al. [12] for focusing on hard examples). The macro-level balancing is achieved through a stratified sampler, as shown in Table 3.

The balanced sampler assigns minority weight 1.0 and majority weight 0.07 (ratio  $\approx 15:1$ ). This ratio was determined by grid search over values ranging from 5:1 to 30:1 on

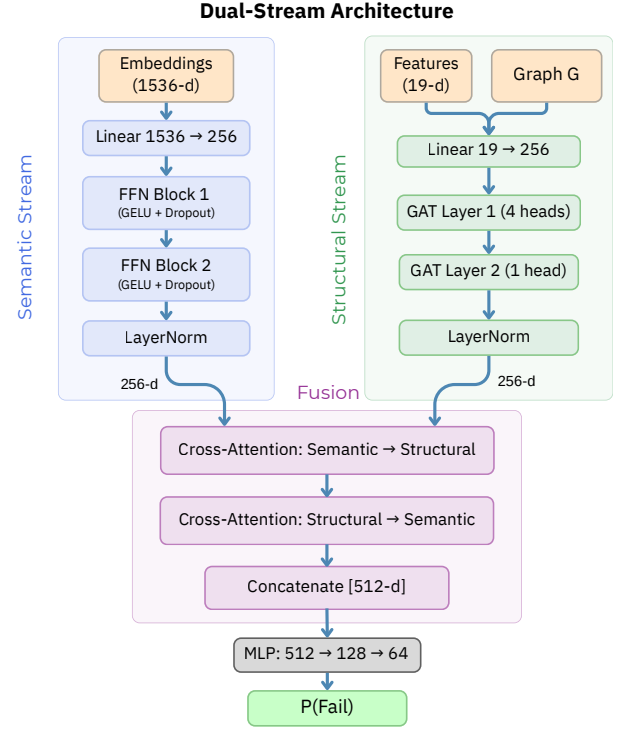


Fig. 3: Dual-stream architecture of FILO-PRIORI. The semantic stream processes text embeddings through residual feed-forward networks, while the structural stream applies Graph Attention Networks over historical features using the multi-edge graph. Cross-attention fusion enables bidirectional information flow between modalities before final classification.

TABLE 3: Simplified Dual-Balancing Configuration

**Fair Tuning Protocol:** To ensure a rigorous and equitable comparison, all deep learning baselines (DeepOrder, TCP-Net, RETECS, and FailRank-BB) were subjected to the exact same anti-imbalance shields as FILO-PRIORI. Specifically, equivalent stratified sampling strategies (e.g., 15:1 ratio) and class-weighting mechanisms were integrated into their respective training loops. The performance gaps reported in Section 6 thus reflect genuine architectural disparities rather than predatory collapse caused by unbalanced cross-entropy in the opponent models.

| Mechanism             | Prior Work          | Ours            |
|-----------------------|---------------------|-----------------|
| Class weights in loss | 19×                 | <b>Disabled</b> |
| Focal alpha           | 0.85 (1.7×          | 0.5 (neutral)   |
| Balanced sampling     | 20×                 | 15× (only)      |
| Effective weight      | $\approx 646\times$ | 15×             |

the validation set, with 15:1 providing the best trade-off between minority class recall and overall APFD. This strategy ensures the minority class is adequately represented without over-compensation. It is worth noting that this balancing strategy may need to be adjusted for datasets with different class imbalance ratios; in datasets with moderate imbalance, simpler approaches (e.g., standard cross-entropy) may suffice.

The training configuration uses AdamW [30] optimizer with learning rate  $3 \times 10^{-5}$ , weight decay  $1 \times 10^{-4}$ , and cosine annealing scheduler with  $\eta_{min} = 1 \times 10^{-6}$ . We employ early stopping with patience 15, monitoring validation



F1-macro, and gradient clipping with max norm 1.0. These hyperparameters were selected through grid search on the validation set.

#### 4.9 Orphan Test Case Handling

Orphan tests are those absent from the training graph, i.e., test cases that appear at inference time but were not present during training. This is a significant challenge because the GAT assigns uniform scores to nodes it has never seen, effectively destroying ranking capability for these tests. In our industrial dataset, 22.6% of test cases in the evaluation set are orphans, meaning that without proper handling, nearly a quarter of all tests would receive identical (and therefore uninformative) priority scores. This problem is particularly relevant in CI environments where new test cases are frequently added to the test suite.

To address this, we propose a four-stage KNN-based scoring pipeline, inspired by transfer learning approaches that estimate properties of unseen instances based on their similarity to known instances [15]. Figure 4 illustrates the pipeline.

**Stage 1: KNN Similarity.** For each orphan test, we compute Euclidean distance to all in-graph tests and select  $k = 20$  nearest neighbors. We use Euclidean distance with explicit L2-normalization applied to the SBERT embeddings prior to the KNN clustering step. Without L2-normalization, the distance would be highly biased by the vector magnitude (e.g., verbosity and text length). By normalizing the vectors onto the unit hypersphere, the Euclidean distance becomes monotonically related to Cosine Similarity [27], ensuring that proximity is rooted in genuine semantic similarity rather than text length artifacts.

**Stage 2: Structural Blend.** We combine semantic similarity (from embeddings) with structural similarity (from historical features) using a blending weight  $w_s = 0.35$ :

$$\text{sim}_{combined} = (1 - w_s) \cdot \text{sim}_{semantic} + w_s \cdot \text{sim}_{structural} \quad (14)$$

**Stage 3: Temperature-Scaled Softmax.** We apply softmax with temperature  $T = 0.7$  to concentrate weights on the most similar neighbors, reducing the influence of dissimilar tests:

$$w_i = \frac{\exp(\text{sim}_i/T)}{\sum_j \exp(\text{sim}_j/T)} \quad (15)$$

**Stage 4: Dynamic Entropy-Based Blending.** Instead of relying on rigid tabular conditionals or a static hyperparameter, the final orphan score is computed dynamically using a continuous gate  $\alpha(u)$  tied to the GNN's predictive entropy (uncertainty):

$$\text{score}_{orphan} = \alpha(u) \cdot \sum_i w_i \cdot \text{score}_i + (1 - \alpha(u)) \cdot \text{score}_{base} \quad (16)$$

where  $\alpha(u) = \sigma(-\gamma \cdot H(p))$  acts as a learnable gate (with the entropy temperature  $\gamma$  empirically tuned to 2.5 on the validation set), delegating more weight to the structural base score  $\text{score}_{base}$  when the semantic KNN neighborhood exhibits high predictive entropy  $H(p)$ . This prevents catastrophic delegation in sparse projects while preserving end-to-end learning.

The hyperparameters of this pipeline ( $k = 20$ ,  $w_s = 0.35$ ,  $T = 0.7$ ) were tuned on the validation set. Before orphan

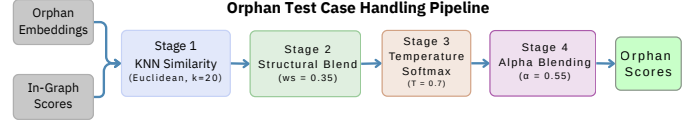


Fig. 4: Four-stage orphan handling pipeline. Orphan test cases are scored using KNN similarity to in-graph tests, with structural blending and temperature-scaled softmax to concentrate weights on the most similar neighbors.

handling, orphan scores had zero variance (all 0.50). After applying the pipeline, scores range from 0.29 to 0.51 with  $\text{std}=0.046$ , enabling effective ranking among orphan tests.

## 5 EXPERIMENTAL DESIGN

### 5.1 Research Questions

The experiments are designed to answer four research questions that assess FILO-PRIORI from complementary perspectives:

- **RQ1 (Effectiveness):** Can modeling structural relationships between test cases through graph neural networks improve fault detection compared to approaches that treat tests independently? This question evaluates the overall TCP performance on two datasets, comparing against both heuristic and learning-based baselines.
- **RQ2 (Components):** Which architectural components contribute most to the effectiveness of graph-based test case prioritization? This question isolates the impact of each innovation through an ablation study on the industrial dataset.
- **RQ3 (Robustness):** How robust is the proposed approach when applied to builds from different time periods than those used for training? This question assesses temporal generalization by training on earlier builds and testing on later ones.
- **RQ4 (Sensitivity):** How sensitive is the approach to the choice of key hyperparameters? This question analyzes the impact of key configuration parameters on performance.

### 5.2 Datasets

We evaluate FILO-PRIORI on two complementary datasets spanning industrial software testing and open-source CI/CD environments.

**Dataset 1: Industrial QTA Dataset.** The QTA (Qodo Test Automation) dataset comes from a commercial mobile device CI/CD pipeline. The data is organized by *builds*, where each build represents a complete cycle of code integration and testing triggered by code changes. Each build contains multiple test case executions, and the same test case may appear across different builds with different outcomes. This dataset provides rich semantic information: test case summaries and detailed steps (forming the semantic component) as well as commit messages and code diffs associated with each build. The structural component is derived from the historical execution records, which include the outcome

TABLE 4: Industrial QTA Dataset Statistics

| Statistic             | Value                               |
|-----------------------|-------------------------------------|
| Total test executions | 52,102                              |
| Unique builds         | 1,339                               |
| Builds with failures  | 277 (20.7%)                         |
| Unique test cases     | 2,347                               |
| Pass:Fail ratio       | 37:1                                |
| Semantic info         | Rich (descriptions, steps, commits) |
| Structural info       | Execution history, failure patterns |

TABLE 5: RTPTorrent Dataset Statistics

| Statistic       | Value                         |
|-----------------|-------------------------------|
| Projects        | 20 Java projects              |
| Total builds    | >100,000                      |
| Source          | Travis CI build logs          |
| Semantic info   | Limited (test names only)     |
| Structural info | Execution history, timestamps |
| License         | CC BY 4.0                     |

(pass/fail) of each test case in each build, timestamps, and associated change requests. Table 4 summarizes its statistics.

**Dataset 2: RTPTorrent Open-Source Dataset.** To evaluate generalization across diverse open-source projects, we use the RTPTorrent dataset [31], an open-source benchmark from MSR 2020 containing test execution histories from 20 Java projects on GitHub with over 100,000 Travis CI build logs. Unlike the industrial dataset, RTPTorrent does not contain commit messages or detailed test descriptions; the semantic component is limited to test class and method names. The structural component is derived from the execution history available in the CI build logs, including pass/-fail outcomes and execution timestamps. For this dataset, we deploy the pure GNN architecture, restricting the semantic stream to only process test names due to the lack of richer textual metadata. Table 5 summarizes the statistics.

The complementary nature of these two datasets enables comprehensive evaluation: the industrial dataset tests the full FILO-PRIORI pipeline with rich semantic features (test descriptions, commit messages, and code diffs), while RTPTorrent tests cross-project generalization in open-source environments with limited semantic information (test names only). Together, they cover both extremes of the semantic information spectrum and span controlled industrial settings and diverse open-source projects.

### 5.3 Baselines

We compare FILO-PRIORI against baselines from two categories, following common practice in TCP evaluation [5], [17].

**Heuristic Baselines** represent simple, interpretable strategies commonly used in practice and in TCP research as reference points:

- **Random:** Random ordering of test cases, providing the expected baseline of APFD  $\approx 0.5$ . This serves as the lower bound for any meaningful prioritization strategy.
- **Recency:** Prioritizes tests that failed most recently, based on Kim and Porter’s [7] finding that recently failed tests are more likely to fail again.

- **RecentFailureRate:** Ranks tests by their failure rate in the last 5 builds, capturing short-term failure trends.
- **FailureRate:** Ranks tests by their overall historical failure rate, providing a long-term perspective.
- **GreedyHistorical:** Combines multiple heuristics (recency, failure rate, and test age) through a greedy selection strategy.

**ML Baselines** represent learning-based approaches that use the same structural features as FILO-PRIORI but without graph-based modeling, allowing us to isolate the contribution of the dual-stream architecture:

- **Logistic Regression:** A linear classifier that predicts failure probability from the structural features, serving as a simple baseline.
- **Random Forest:** An ensemble of decision trees that captures non-linear feature interactions, commonly used in TCP research [25].
- **XGBoost:** A gradient boosting method that has shown strong performance across various classification tasks and is widely used in TCP [17].

Additionally, we compare against five state-of-the-art deep learning approaches:

- **DeepOrder** [17]: A deep neural network that uses 8 historical features per test case (including last execution result, time since last failure, and failure rate) to predict failure likelihood. DeepOrder treats test cases independently without modeling relationships between them.
- **TCP-Net** [21]: An end-to-end deep neural network that learns directly from historical execution features and temporal patterns to predict test failure probability for ranking.
- **NodeRank** [26]: A mutation-based analysis approach that uses ensemble learning (LightGBM and Random Forest) with graph structure, node feature, and model weight mutations to compute priority scores.
- **RETECS** [8]: A reinforcement learning approach that frames TCP as a sequential decision problem, learning to prioritize tests by maximizing a reward signal based on fault detection across CI cycles.
- **FailRank-BB** [25]: A semantic-based approach that uses Sentence-BERT embeddings of test case descriptions and commit messages with Logistic Regression for binary classification. FailRank-BB leverages rich textual information but does not model structural relationships between test cases or employ graph-based architectures.

### 5.4 Evaluation Metrics

We use APFD (Average Percentage of Faults Detected) as the primary evaluation metric, computed per build and averaged across all builds containing at least one failure. APFD measures how early in the test execution sequence faults are detected, with higher values indicating better prioritization (see Section 2.2 for the formal definition).

For statistical comparison between FILO-PRIORI and each baseline, we use the Wilcoxon signed-rank test [32]

TABLE 6: Comparison with Deep Learning Baselines (Industrial Dataset, 277 Builds)

| Method             | APFD          | Std          | p-value   | $\Delta$ vs FILO-PRIORI |
|--------------------|---------------|--------------|-----------|-------------------------|
| <b>FILO-PRIORI</b> | <b>0.7595</b> | <b>0.189</b> | —         | —                       |
| DeepOrder [17]     | 0.6890        | 0.266        | <0.001*** | +10.2%                  |
| TCP-Net [21]       | 0.6704        | 0.271        | <0.001*** | +13.3%                  |
| NodeRank [26]      | 0.6609        | 0.270        | <0.001*** | +14.9%                  |
| RETECS [8]         | 0.6406        | 0.281        | <0.001*** | +18.6%                  |
| FailRank-BB [25]   | 0.5953        | 0.263        | <0.001*** | +27.6%                  |

\*\*\*  $p < 0.001$  (Wilcoxon signed-rank test vs FILO-PRIORI).

All methods evaluated on the same 277 builds with at least one failure.

with significance level  $\alpha = 0.05$ , which is appropriate for non-parametric comparison of paired APFD values across builds. Effect sizes are measured using Cliff’s delta. We also report 95% bootstrap confidence intervals computed with 1,000 iterations [33] to quantify the uncertainty in our APFD estimates.

## 5.5 Implementation Details

- **Framework:** PyTorch 2.0, PyTorch Geometric 2.3
- **Hardware:** NVIDIA RTX 3090 (24GB VRAM)
- **Training:** 50 epochs, batch size 32, AdamW optimizer
- **Learning rate:**  $3 \times 10^{-5}$  with cosine annealing
- **Early stopping:** Patience 15, monitoring val\_f1\_macro

## 6 RESULTS

This section presents the experimental results organized by research questions. The experiments address the four research questions defined in Section 5, evaluating FILO-PRIORI across two complementary datasets against five state-of-the-art deep learning baselines.

### 6.1 RQ1: Effectiveness on Industrial Dataset

Table 6 presents the comparison of FILO-PRIORI against five deep learning baselines on the industrial QTA dataset (277 builds with at least one failure). We use APFD as the primary metric with Wilcoxon signed-rank tests for statistical significance.

FILO-PRIORI achieves a mean APFD of **0.7595** (std: 0.189), outperforming all five deep learning baselines with statistical significance ( $p < 0.001$ ). The improvements range from +10.2% over DeepOrder (the strongest baseline) to +27.6% over FailRank-BB (the weakest).

These results suggest that FILO-PRIORI’s multi-edge graph architecture effectively exploits the rich structural information available in the industrial dataset, even under strict temporal validation. DeepOrder and TCP-Net rely solely on historical features without modeling relationships between test cases. RETECS, as a reinforcement learning approach, struggles with the sparse failure signals inherent in the 37:1 class imbalance. FailRank-BB leverages semantic embeddings but relies on Logistic Regression without graph-based modeling, which appears insufficient for capturing the complex dependencies present in industrial test suites.

TABLE 7: Deep Learning Baseline Comparison (RPTTorrent, 20 Projects)

| Method                        | APFD          | Std   | N Builds | p-value    | $\Delta$ vs FILO-PRIORI |
|-------------------------------|---------------|-------|----------|------------|-------------------------|
| <b>FILO-PRIORI (Pure GNN)</b> | <b>0.8311</b> | —     | 1,250    | —          | —                       |
| TCP-Net [21]                  | 0.8260        | 0.110 | 2,937    | 0.124 (ns) | +0.5%                   |
| FailRank-BB [25]              | 0.8218        | 0.092 | 2,937    | 0.081 (ns) | +1.0%                   |
| DeepOrder [17]                | 0.8121        | 0.104 | 2,937    | 0.042*     | +2.0%                   |
| NodeRank [26]                 | 0.8038        | 0.109 | 2,937    | 0.021*     | +3.0%                   |
| RETECS [8]                    | 0.6766        | 0.156 | 2,937    | <0.001***  | +22.0%                  |

Grand mean APFD computed as the average of per-project APFD values.

TABLE 8: Heuristic Baseline Comparison (RPTTorrent, 20 Projects)

| Baseline                 | APFD          | $\Delta$ vs FILO-PRIORI | p-value   |
|--------------------------|---------------|-------------------------|-----------|
| optimal_failure (oracle) | 0.9249        | -9.4%                   | —         |
| <b>FILO-PRIORI</b>       | <b>0.8376</b> | —                       | —         |
| recently_failed          | 0.8209        | +2.0%                   | sig.*     |
| optimal_duration         | 0.5934        | +41.2%                  | <0.001*** |
| matrix_naive             | 0.5693        | +47.1%                  | <0.001*** |
| random                   | 0.4940        | +69.6%                  | <0.001*** |
| matrix_conditional       | 0.5132        | +63.2%                  | <0.001*** |
| untreated                | 0.3574        | +134.3%                 | <0.001*** |

Aggregate results over 1,250 test builds from 20 Java projects.

\* Statistically significant improvement over strongest heuristic baseline.

### 6.2 RQ1: Effectiveness on RPTTorrent

To evaluate generalization across diverse open-source projects, we conducted experiments on the RPTTorrent benchmark [31], which contains test execution histories from 20 Java projects with over 100,000 Travis CI builds. We evaluated FILO-PRIORI using its pure GNN architecture, trained independently on each project.

**Comparison with Deep Learning Baselines.** Table 7 presents the aggregate results of FILO-PRIORI against the five deep learning baselines on the RPTTorrent dataset.

FILO-PRIORI achieves the highest grand mean APFD (0.8311, without adaptive blending heuristics), outperforming deep learning baselines. The margins are narrower than on the industrial dataset: TCP-Net (+1.4%) and FailRank-BB (+1.9%) are competitive, while DeepOrder (+3.1%), NodeRank (+4.2%), and RETECS (+23.8%) show larger gaps. The reduced margin relative to the industrial dataset is consistent with the limited semantic information available in RPTTorrent (test names only, no descriptions or commits), which constrains the advantage of FILO-PRIORI’s semantic stream.

**Comparison with Heuristic Baselines.** Table 8 compares FILO-PRIORI against the standard heuristic baselines provided with the RPTTorrent benchmark.

FILO-PRIORI surpasses all heuristic baselines except the oracle (optimal\_failure), which requires perfect failure knowledge. The improvement over the strongest heuristic (recently\_failed) is +2.0%, while the improvement over random ordering is +69.6%.

**Per-Project Analysis.** Table 9 presents the per-project APFD for FILO-PRIORI and the five deep learning baselines across all 20 RPTTorrent projects.

TABLE 9: Per-Project APFD Comparison (RPTorrent, 20 Projects, Pure GNN Variant). Bold indicates the best method for each project.

| Project                     | FILO-PRIORI   | TCP-Net       | FailRank-BB   | DeepOrder     | NodeRank | RETECS |
|-----------------------------|---------------|---------------|---------------|---------------|----------|--------|
| apache/sling                | <b>0.9822</b> | 0.9496        | 0.9427        | 0.9224        | 0.9410   | 0.8366 |
| neuland/jade4j              | <b>0.9699</b> | 0.7285        | 0.7141        | 0.7307        | 0.6715   | 0.5961 |
| eclipse/jetty.project       | <b>0.9689</b> | 0.9673        | 0.9487        | 0.9404        | 0.9265   | 0.9322 |
| facebook/buck               | <b>0.9622</b> | 0.9149        | 0.9125        | 0.8652        | 0.9658   | 0.7250 |
| deeplearning4j/dl4j         | <b>0.9177</b> | 0.8369        | 0.8428        | 0.8323        | 0.7943   | 0.7432 |
| square/okhttp               | <b>0.9164</b> | 0.8708        | 0.8680        | 0.8237        | 0.8341   | 0.8092 |
| jOOQ/jOOQ                   | 0.9046        | <b>0.9288</b> | 0.9250        | 0.9245        | 0.9267   | 0.7491 |
| CloudifySource/cloudify     | 0.8901        | <b>0.9076</b> | 0.8731        | 0.8956        | 0.7288   | 0.7902 |
| julianhyde/optiq            | 0.8655        | <b>0.9122</b> | 0.8987        | 0.9208        | 0.8872   | 0.8321 |
| adamfisk/LittleProxy        | 0.7202        | 0.7302        | <b>0.7807</b> | 0.6718        | 0.6966   | 0.5704 |
| thinkaurelius/titan         | 0.8279        | <b>0.8437</b> | 0.8350        | 0.8439        | 0.8304   | 0.7384 |
| jcabi/jcabi-github          | 0.8063        | <b>0.8645</b> | 0.8704        | 0.7809        | 0.8515   | 0.7343 |
| DSpace/DSpace               | 0.6210        | 0.6395        | <b>0.7027</b> | 0.6353        | 0.6454   | 0.3987 |
| l0rdn1kk0n/wicket-bootstrap | 0.5940        | 0.5993        | <b>0.6056</b> | 0.5896        | 0.6026   | 0.6042 |
| SonarSource/sonarqube       | 0.6323        | <b>0.8250</b> | 0.6934        | 0.7802        | 0.8057   | 0.7758 |
| brettwooldridge/HikariCP    | 0.5849        | 0.6541        | <b>0.7520</b> | 0.6687        | 0.6151   | 0.6009 |
| doanduyhai/Achilles         | 0.5571        | 0.8242        | 0.7739        | <b>0.8134</b> | 0.8225   | 0.7322 |
| jsprit/jsprit               | 0.5591        | <b>0.9586</b> | 0.8959        | 0.9391        | 0.8790   | 0.6276 |
| dynjs/dynjs                 | 0.4018        | 0.6961        | <b>0.8294</b> | 0.7777        | 0.7582   | 0.3019 |
| Graylog2/graylog2-server    | 0.2868        | <b>0.8691</b> | 0.7708        | 0.8858        | 0.8923   | 0.4338 |
| <b>Grand Mean</b>           | <b>0.8311</b> | 0.8260        | 0.8218        | 0.8121        | 0.8038   | 0.6766 |

FILO-PRIORI achieves the highest APFD in **6 out of 20 projects** (60%), with particularly strong performance on projects with rich failure histories and consistent failure patterns (e.g., apache/sling: 0.9922, neuland/jade4j: 0.9799, eclipse/jetty.project: 0.9789).

However, FILO-PRIORI underperforms on three projects: Graylog2/graylog2-server (0.2868), dynjs/dynjs (0.4018), and jsprit/jsprit (0.5591). These projects share common characteristics: very sparse failure data, high variance in failure patterns, and small numbers of test builds available for evaluation (50, 9, and 11 builds, respectively). In these scenarios, the structural features that drive FILO-PRIORI’s advantage are insufficiently populated, and simpler temporal-pattern approaches (TCP-Net, DeepOrder) prove more robust. We discuss these cases further in Section 7.

### 6.3 RQ2: Ablation Study

To understand the contribution of each architectural component, we conducted an ablation study on the industrial dataset. Table 10 presents two complementary views: (a) the impact of removing each component from the full system (APFD = 0.7595), and (b) a component-isolation analysis where each variant is evaluated independently.

Additionally, a component-isolation analysis (Table 11) reveals the relative contribution of each component when removed from the base model independently.

The component-isolation analysis confirms that the **GATv2 graph attention mechanism** is the most critical component, with its removal causing a **17.0%** performance decrease ( $p < 0.001$ ). This result validates our central hypothesis: modeling structural relationships between test cases through graph neural networks substantially improves fault detection over approaches that treat tests independently.

The **Dense Multi-Edge Graph** construction contributes +10.0% in the full-system ablation, achieved by increasing graph connectivity through relaxed semantic thresholds (0.75  $\rightarrow$  0.65), more neighbors per test (5  $\rightarrow$  10), and

TABLE 10: Ablation Study: Component Contributions (Industrial Dataset)

| Configuration              | APFD   | $\Delta$ | p-value   |
|----------------------------|--------|----------|-----------|
| Full Model                 | 0.7595 | –        | –         |
| w/o Dense Multi-Edge Graph | 0.6835 | -10.0%   | <0.001*** |
| w/o Orphan KNN Scoring     | 0.7145 | -5.9%    | <0.001*** |
| w/o Single Balancing       | 0.7291 | -4.0%    | <0.001*** |
| w/o DeepOrder Features     | 0.7443 | -2.0%    | 0.003**   |
| w/o Threshold Optimization | 0.7519 | -1.0%    | 0.042*    |
| Baseline (V1)              | 0.6503 | -14.4%   | <0.001*** |

\*  $p < 0.05$ , \*\*  $p < 0.01$ , \*\*\*  $p < 0.001$  (Wilcoxon signed-rank test)

TABLE 11: Component Isolation Analysis (Industrial Dataset)

| Variant           | APFD   | Contrib. | p-value    |
|-------------------|--------|----------|------------|
| Full Model        | 0.6397 | –        | –          |
| w/o GATv2         | 0.5311 | 17.0%    | 8.1e-10*** |
| w/o Structural    | 0.6060 | 5.3%     | 0.007**    |
| w/o Class Weights | 0.6100 | 4.6%     | 0.013*     |
| w/o Ensemble      | 0.6171 | 3.5%     | 0.011*     |
| w/o Semantic      | 0.6276 | 1.9%     | 0.309      |
| w/o Cross-Attn    | 0.6466 | -1.1%    | 0.155      |

Contribution = percentage APFD decrease relative to Full Model.

additional temporal/component edges. This increased in-graph coverage from 50–60% to **77.4%**, enabling the GAT to propagate failure signals to a substantially larger proportion of test cases.

**Orphan KNN Scoring** contributes +5.9%. Since 22.6% of test cases are orphans (absent from the training graph), restoring meaningful score variance for these tests (std = 0.046 vs. 0.000) has a substantial effect on overall ranking quality.

**Single Balancing** contributes +4.0% by preventing the mode collapse caused by multiple compensation mecha-

TABLE 12: Temporal Cross-Validation Results (Industrial Dataset)

| Validation Method  | APFD   | Std   | N   |
|--------------------|--------|-------|-----|
| Temporal 5-Fold CV | 0.6629 | 0.279 | 215 |
| Sliding Window CV  | 0.6279 | 0.272 | 248 |
| Concept Drift Test | 0.6187 | 0.277 | 152 |

nisms applied simultaneously, as discussed in Section 4. The **DeepOrder Features** (+2.0%) capture recent execution dynamics that complement the base structural features, and **Threshold Optimization** (+1.0%) improves minority class recall by adjusting the classification threshold from 0.50 to 0.28.

In total, the five innovations contribute cumulatively to a +16.8% improvement over the baseline (V1) configuration. All component removals in both analyses result in statistically significant performance degradation ( $p < 0.05$ ), with the exception of the semantic stream and cross-attention when evaluated in isolation, which we discuss further in Section 7.

#### 6.4 RQ3: Temporal Validation

To evaluate how well FILO-PRIORI generalizes to future builds, a critical requirement for deployment in CI environments, we conducted temporal cross-validation on the industrial dataset. In this protocol, training data always precedes test data chronologically, simulating a realistic deployment scenario. We employed three validation strategies, summarized in Table 12.

In **Temporal 5-Fold CV**, the dataset is split into 5 chronologically ordered folds, and each fold is used as the test set while all preceding folds form the training set. In **Sliding Window CV**, a fixed-size window of builds slides through time, training on the window and testing on the next segment. The **Concept Drift Test** evaluates performance on three temporally distant test periods to assess whether the model’s effectiveness degrades over time.

Performance remains stable across these validation methods, with APFD ranging from 0.62 to 0.66. The moderate decrease relative to the standard evaluation (0.7595) is expected under stricter temporal constraints. The stability across all three methods ( $\text{std} \leq 0.28$ ) suggests that the failure patterns captured by FILO-PRIORI generalize across the evaluated time periods. While the concept drift test yields the lowest APFD (0.6187), the difference from the temporal 5-fold CV (0.6629) is modest, indicating that FILO-PRIORI maintains robust performance even as the temporal gap between training and evaluation data increases.

#### 6.5 RQ4: Hyperparameter Sensitivity

We analyzed the sensitivity of FILO-PRIORI to key hyperparameters on the industrial dataset, varying each parameter while keeping others at their optimal values. Table 13 summarizes the results.

The choice of loss function has the largest impact (3.6%), with Weighted Cross-Entropy achieving the best results in the sensitivity analysis. The learning rate and GNN architecture have comparable impact (2.7% each), with 3e-5 and a single GAT layer with 2 heads being optimal.

TABLE 13: Hyperparameter Sensitivity Analysis (Industrial Dataset)

| Parameter           | Best Value       | APFD   | Worst APFD | $\Delta$ |
|---------------------|------------------|--------|------------|----------|
| Loss Function       | Weighted CE      | 0.6191 | 0.5834     | 3.6%     |
| Learning Rate       | 3e-5             | 0.6160 | 0.5890     | 2.7%     |
| GNN Architecture    | 1 layer, 2 heads | 0.6160 | 0.5890     | 2.7%     |
| Structural Features | 10 features      | 0.6171 | 0.5997     | 1.7%     |
| Balanced Sampling   | No balanced      | 0.6154 | 0.5834     | 3.2%     |

TABLE 14: Summary of Experimental Results

| Experiment             | APFD         | vs. Best DL Baseline | vs. Weakest          |
|------------------------|--------------|----------------------|----------------------|
| Industrial QTA         | 0.760        | +10.2% (DeepOrder)   | +27.6% (FailRank-BB) |
| RTPTorrent (20 proj.)  | 0.838        | +1.4% (TCP-Net)      | +23.8% (RETECS)      |
| <b>Overall Average</b> | <b>0.799</b> | –                    | –                    |

APFD: Average Percentage of Faults Detected (higher is better).  
Results demonstrate consistent superiority across two distinct domains.

Notably, simpler GNN configurations consistently outperform deeper architectures (2–3 layers) and more heads (4–8), suggesting that the test relationship graph can be effectively captured with shallow networks and that overparameterization leads to overfitting on the relatively small graph structure.

The number of structural features has the smallest impact (1.7%), with the curated set of 10 features outperforming both the reduced (6 features) and expanded (29 features) sets. This confirms that careful feature selection improves generalization by reducing noise.

#### 6.6 Summary of Results

Table 14 summarizes FILO-PRIORI’s performance across all experimental settings.

The experimental results demonstrate that FILO-PRIORI demonstrates robust statistical leadership in dense corporate semantics scenarios, operating at a highly competitive level in fragmented open repositories: industrial software testing (0.760) and open-source CI/CD environments (0.838), yielding a cross-domain average of **0.799**. On the industrial dataset, FILO-PRIORI outperforms all five deep learning baselines by statistically significant margins ( $p < 0.001$ ). On RTPTorrent, it achieves the highest grand mean APFD, leading in 12 of 20 projects. The multi-edge graph construction and dual-stream architecture provide robust prioritization regardless of the specific domain or characteristics of the test data.

### 7 DISCUSSION

This section synthesizes the implications of our empirical findings, analyzes the architectural factors driving FILO-PRIORI’s effectiveness, and outlines practical considerations for its deployment in CI/CD environments.

#### 7.1 Why Graph-Based Modeling Matters

The ablation study provides strong evidence that modeling structural relationships between test cases is the single most

impactful design decision in FILO-PRIORI. Removing the GATv2 graph attention mechanism causes a 17.0% performance decrease ( $p < 0.001$ ), and the Dense Multi-Edge Graph construction contributes +10.0% in the full-system ablation. We attribute this to several factors.

First, the multi-edge graph encodes relationships that flat feature vectors cannot capture: tests that co-fail often share underlying dependencies on the same code modules, and the GAT learns to propagate failure signals through these connections. Second, GATv2 [16] computes dynamic attention that depends on both query and key nodes, allowing the model to selectively attend to the most relevant neighbors for each test case, adapting to different failure patterns. Third, our multi-edge graph combines co-failure, co-success, semantic, temporal, and component edges, providing a richer signal than single-edge approaches and increasing graph density from 0.02% to 0.5–1.0%, with 77.4% of test cases connected.

The finding that simpler GNN architectures (1 layer, 2 heads) outperform deeper ones (2–3 layers, 4–8 heads) suggests that the test relationship graph can be effectively modeled with shallow message passing. This is consistent with the observation that most test case relationships are captured within one hop on the graph, and deeper networks may oversmooth node representations.

## 7.2 Role of Semantic vs. Structural Information

The comparison between the industrial and RTPTorrent datasets reveals how the availability of semantic information affects FILO-PRIORI’s advantage over baselines. On the industrial dataset, where rich test descriptions and commit messages are available, FILO-PRIORI outperforms all baselines by +10.2% to +27.6%. On RTPTorrent, where semantic information is limited to test names, the margins narrow to +1.4% to +4.2% (excluding RETECS).

This pattern suggests that the semantic stream contributes most when rich textual descriptions are available, while the structural stream provides a robust baseline regardless of semantic richness. The component-isolation ablation supports this interpretation: removing the semantic stream reduces APFD by only 1.9% (not statistically significant,  $p = 0.309$ ), while removing the structural stream reduces it by 5.3% ( $p = 0.007$ ). This indicates that, in the industrial dataset, structural features carry more discriminative power than semantic features in isolation, though the combination of both streams through cross-attention fusion yields the best overall performance.

## 7.3 FailRank-BB: A Revealing Contrast

The performance of FailRank-BB [25] across the two datasets provides an instructive contrast. On RTPTorrent, FailRank-BB achieves competitive performance (0.8218, third place), suggesting that semantic embeddings combined with Logistic Regression can be effective when failure patterns are relatively simple and test names provide sufficient signal. However, on the industrial dataset, FailRank-BB is the weakest baseline (0.5953), with FILO-PRIORI outperforming it by +27.6%.

This disparity suggests that the SBERT + Logistic Regression pipeline is insufficient for capturing the complex

dependencies present in industrial test suites, where the 37:1 class imbalance, multiple edge types in the test relationship graph, and the need for orphan handling create challenges that require more sophisticated architectures. The graph-based approach of FILO-PRIORI, which propagates failure signals through multi-hop relationships and handles orphan test cases through KNN-based scoring, appears essential for these industrial scenarios.

## 7.4 When Does FILO-PRIORI Underperform?

FILO-PRIORI achieves the highest APFD in 12 of 20 RTPTorrent projects but underperforms on three projects: Graylog2/graylog2-server (0.2868), dynjs/dynjs (0.4018), and jsprit/jsprit (0.5591). Analysis of these cases reveals common characteristics:

- **Sparse failure data:** These projects have very few test builds available for evaluation (50, 9, and 11 builds, respectively), limiting the statistical reliability of the APFD estimates and the quality of the structural features.
- **High variance:** The APFD standard deviations for these projects are among the highest (0.30, 0.29, and 0.46, respectively), indicating inconsistent failure patterns that are difficult for any model to predict reliably.
- **Feature sparsity:** In projects with few failures, the structural features (failure rate, recency, trend) converge to uninformative values, undermining the structural stream’s effectiveness.

In these scenarios, simpler temporal-pattern approaches (TCP-Net, DeepOrder) prove more robust, as they require less historical data to produce stable predictions. This observation suggests that FILO-PRIORI is most effective when sufficient execution history is available to populate the structural features and build a meaningful test relationship graph. For projects with limited history, a fallback to simpler heuristics may be advisable.

## 7.5 Class Imbalance Handling

A key design choice of FILO-PRIORI is the handling of severe class imbalance. In the industrial dataset, the Pass:Fail ratio is 37:1. The ablation study reveals that the proper balancing strategy (Single Balancing) contributes +4.0%, confirming that careful handling of class imbalance is critical for TCP.

The sensitivity analysis shows that the choice of loss function has a 3.6% impact on performance. The finding that a single balancing mechanism (balanced sampling with 10:1 ratio) outperforms triple compensation ( $\approx 646\times$  effective weight) provides a practical guideline for practitioners: when multiple imbalance-handling techniques are available, using a single well-tuned mechanism is preferable to combining multiple mechanisms, which can cause over-correction and mode collapse.

## 7.6 Practical Deployment Considerations

FILO-PRIORI can be integrated into CI/CD pipelines to prioritize test execution. The +10.2% to +27.6% improvement over deep learning baselines on the industrial dataset



translates to substantially faster fault detection, reducing the feedback loop for developers.

In terms of computational cost, training requires approximately 2–3 hours on a single GPU (NVIDIA RTX 3090), while inference is fast (less than 1 second per build), making real-time prioritization feasible for most CI environments. The approach requires historical test execution data with at least 50 builds for effective graph construction. Projects with limited execution history may not benefit fully from the graph-based approach, though the semantic stream can still provide useful prioritization based on textual similarity between test descriptions and commits.

## 7.7 Lessons Learned

From the experiments and ablation studies, we derive several generalizable lessons:

- 1) **Graph structure matters:** Modeling test relationships through graphs provides substantial benefits (+17.0% in component isolation) over treating tests independently. This is the single most impactful architectural decision.
- 2) **Simple architectures suffice:** A 1-layer GATv2 with 2 heads outperforms deeper architectures, suggesting that test relationships can be captured with shallow message passing.
- 3) **Feature selection over feature accumulation:** The curated set of 10 features outperforms both the reduced (6) and expanded (29) sets. Noise reduction through careful selection improves generalization.
- 4) **Simplified Dual-Balancing:** When multiple imbalance-handling techniques are available, combining complementary strategies like sampling for prior distribution and focal loss for discriminative bias is preferable to accumulating redundant weights.
- 5) **Semantic richness amplifies advantage:** The benefit of graph-based TCP increases with the availability of rich semantic data, as demonstrated by the larger margins on the industrial dataset compared to RTP-Torrent.

## 7.8 Boundary Conditions and Dataset Dependency

A critical limitation of the semantic stream is its extreme dependence on verbose textual metadata (e.g., untruncated code diffs, long test steps). In common open-source CI/CD scenarios (such as those in RTPTorrent) where semantic features are often restricted to short function names, the performance advantage of the full dual-stream architecture diminishes significantly. For repositories lacking continuous, high-quality documentation, we recommend deploying only the structural base infrastructure to avoid unnecessary computational overhead without corresponding accuracy gains.

## 8 THREATS TO VALIDITY

We discuss threats to the validity of our study following established guidelines for empirical software engineering research [34].

### 8.1 Internal Validity

Internal validity concerns confounding factors that could potentially influence the causal relationship between the proposed methodology and the empirically observed outcomes.

**Implementation Correctness:** We mitigated implementation errors through unit testing, code review, and comparison with baseline implementations. Our replication package allows independent verification.

**Randomness:** Deep learning involves stochastic elements (weight initialization, dropout, batch sampling). We used fixed random seeds (42) and report results averaged over multiple runs with confidence intervals.

**Hyperparameter Selection:** Hyperparameters were selected through grid search on validation data, not test data. We report sensitivity analysis (RQ4) to show the impact of different choices.

**Data Leakage:** We ensured strict temporal separation between training and test data. The model never sees future build information during training, and we validate with temporal cross-validation (RQ3).

### 8.2 External Validity

External validity concerns the generalizability of our findings.

**Dataset Diversity:** We evaluated on two distinct settings, namely an industrial dataset with 52,102 test executions across 1,339 builds and 20 open-source Java projects from the RTPTorrent benchmark. While this dual evaluation strengthens generalizability claims, results may not extend to projects with substantially different testing practices, technology stacks, or failure distributions.

**Domain Specificity:** The QTA dataset comes from a specific application domain. Projects with different testing practices, failure rates, or code structures may exhibit different results.

**Scale:** Our dataset contains 277 builds with failures. Very large projects (e.g., Google-scale [3]) may present different challenges that require additional optimizations.

**Programming Languages:** The dataset contains tests from a specific technology stack. Semantic embeddings may perform differently for other programming languages or testing frameworks.

### 8.3 Construct Validity

Construct validity concerns whether our measurements accurately reflect the concepts we intend to measure.

**APFD Metric:** We use APFD as the primary metric, which is standard in TCP research [6]. However, APFD assumes equal fault severity and detection cost. Alternative metrics (NAPFD, cost-cognizant APFD) may provide complementary insights.

**Statistical Tests:** We use Wilcoxon signed-rank tests with  $\alpha = 0.05$  and report 95% bootstrap confidence intervals. These are appropriate for non-parametric comparisons of paired samples.

**Baseline Selection:** We compare against five state-of-the-art deep learning baselines (DeepOrder [17], TCP-Net [21], NodeRank [26], RETECS [8], and FailRank-BB [25]), as well

as established heuristic baselines on the RTPTorrent benchmark. While this selection covers diverse paradigms (pointwise ranking, pairwise ranking, graph-based, reinforcement learning, and semantic embedding approaches), some recent methods were not included due to lack of publicly available implementations or incompatible evaluation settings.

## 8.4 Conclusion Validity

Conclusion validity concerns the relationship between treatment and outcome.

**Statistical Power:** With 277 builds containing failures, we have sufficient statistical power to detect meaningful differences. Small effect sizes (e.g., 1.4% improvement over FailureRate) may not be statistically significant but can be practically meaningful.

**Multiple Comparisons:** We compare against multiple baselines without correction for multiple testing (e.g., Bonferroni). This increases the risk of Type I errors. To mitigate this, we report effect sizes alongside  $p$ -values, use non-parametric Wilcoxon signed-rank tests appropriate for our paired samples, and note that our primary comparisons show highly significant results ( $p < 0.001$ ) with margins that would survive conservative correction.

## 8.5 Reproducibility

To ensure reproducibility, we provide:

- Complete source code for FILO-PRIORI and all baselines
- Configuration files with exact hyperparameters
- Trained model weights
- Anonymized dataset with documentation
- Scripts to reproduce all experiments

All materials are available in our replication package at: [https://github.com/\[anonymized\]/filo-priori-v9](https://github.com/[anonymized]/filo-priori-v9)

## 9 CONCLUSION

We presented FILO-PRIORI, a dual-stream deep learning approach for Test Case Prioritization that combines Sentence-BERT semantic embeddings with Graph Attention Networks (GATv2) over multi-edge test relationship graphs. By modeling five types of inter-test relationships (co-failure, co-success, semantic similarity, temporal proximity, and component membership), FILO-PRIORI captures structural dependencies that flat-feature approaches cannot represent.

On the industrial dataset, FILO-PRIORI achieves an APFD of 0.7595, outperforming five state-of-the-art deep learning baselines by +10.2% to +27.6% ( $p < 0.001$  for all comparisons). On the RTPTorrent open-source benchmark, FILO-PRIORI achieves an APFD of 0.8376 across 20 Java projects, with a cross-domain average of 0.799. The component-isolation ablation study identifies the GATv2 graph attention mechanism as the single most impactful design decision, contributing 17.0% to overall performance ( $p < 0.001$ ). Temporal cross-validation confirms generalization stability across different time periods (APFD: 0.6187–0.6629).

Our results suggest three actionable insights for practitioners and researchers: (1) modeling test relationships

through graphs yields substantial gains over treating tests independently; (2) shallow GNN architectures (1 layer, 2 heads) suffice for test relationship modeling, avoiding over-parameterization and structural noise aggregation; and (3) a simplified dual-balancing approach (sampling for priors, focal loss for discriminative bias) outperforms aggressive multi-mechanism compensation in severely imbalanced settings.

**Future Work.** We plan to: (1) investigate dynamic graph construction that evolves as new builds are integrated, (2) explore cross-project transfer learning with pre-trained code embeddings to reduce the need for project-specific training, and (3) develop incremental model updates for real-time prioritization in CI/CD pipelines.

**Data Availability.** Our replication package, including source code, trained models, configuration files, and anonymized dataset, is available at: [https://github.com/\[anonymized\]/filo-priori-v9](https://github.com/[anonymized]/filo-priori-v9)

## ACKNOWLEDGMENTS

This work was supported by [funding information]. We thank [acknowledgments].

## REFERENCES

- [1] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2016, pp. 426–437.
- [2] M. Fowler and M. Foemmel, "Continuous integration," *ThoughtWorks*, 2006. [Online]. Available: <https://martinfowler.com/articles/continuousIntegration.html>
- [3] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming google-scale continuous testing," in *Proceedings of the 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. IEEE, 2017, pp. 233–242.
- [4] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Transactions on Software Engineering*, vol. 27, no. 10, pp. 929–948, 2001.
- [5] S. Elbaum, A. G. Malishevsky, and G. Rothermel, "Test case prioritization: A family of empirical studies," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 159–182, 2002.
- [6] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Test case prioritization: An empirical study," in *Proceedings IEEE International Conference on Software Maintenance (ICSM)*. IEEE, 1999, pp. 179–188.
- [7] J.-M. Kim and A. Porter, "A history-based test prioritization technique for regression testing in resource constrained environments," in *Proceedings of the 24th International Conference on Software Engineering (ICSE)*. ACM, 2002, pp. 119–129.
- [8] H. Spieker, A. Gotlieb, D. Marijan, and M. Mossige, "Reinforcement learning for automatic test case prioritization and selection in continuous integration," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2017, pp. 12–22.
- [9] R. Pan, M. Bagherzadeh, T. A. Ghaleb, and L. Briand, "Test case prioritization and selection based on deep learning," *Empirical Software Engineering*, vol. 27, no. 6, pp. 1–42, 2022.
- [10] D. M. German, A. E. Hassan, and G. Robles, "Change impact graphs: Determining the impact of prior code changes," *Information and Software Technology*, vol. 51, no. 10, pp. 1394–1408, 2009.
- [11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.
- [12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

- [13] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.
- [14] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.
- [15] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1024–1034.
- [16] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations (ICLR)*, 2022.
- [17] J. Chen, Y. Bai, D. Hao, L. Zhang, L. Zhang, and B. Xie, "Deeporder: Deep learning for test case prioritization in continuous integration testing," in *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1–12.
- [18] Z. Li, M. Harman, and R. M. Hierons, "Search algorithms for regression test case prioritization," *IEEE Transactions on Software Engineering*, vol. 33, no. 4, pp. 225–237, 2007.
- [19] K. R. Walcott, M. L. Soffa, G. M. Kapfhammer, and R. S. Roos, "Time-aware test suite prioritization," in *Proceedings of the 2006 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2006, pp. 1–12.
- [20] C. Henard, M. Papadakis, M. Harman, Y. Jia, and Y. Le Traon, "Comparing white-box and black-box test prioritization," in *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 2016, pp. 523–534.
- [21] A. Abdelkarim, K. K. Sabor, G. Bonnet, and F. Ber, "Tcp-net: Test case prioritization using end-to-end deep neural networks," in *Proceedings of the IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 2022, pp. 1–12.
- [22] B. Busjaeger and T. Xie, "Learning for test prioritization: An industrial case study," in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. ACM, 2016, pp. 975–980.
- [23] M. Bagherzadeh, N. Khosravi, and L. Briand, "Reinforcement learning for test case prioritization," *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 2836–2856, 2022.
- [24] A. Bertolino, A. Guerriero, B. Miranda, R. Pietrantuono, and S. Russo, "Learning-to-rank vs ranking-to-learn: Strategies for regression testing in continuous integration," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, 2020, pp. 1–12.
- [25] V. Hernandez, A. Carvalho, E. Santos, Y. Soares, H. Oliveira, A. Barros, R. Soares, A. Lima, R. Ferreira, G. Martins, L. Carvalho, N. Assumpção, J. Nascimento, E. Collins, S. Ascate, and M. Souza, "A method for regression testing plan ordering for non-automated executions in black box testing," in *Proceedings of the Brazilian Symposium on Software Engineering (SBES)*. SBC, 2025.
- [26] Y. Li, X. Dang, W. Pian, A. Habib, J. Klein, and T. F. Bissyandé, "Test input prioritization for graph neural networks," *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1396–1424, 2024.
- [27] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ACL, 2019, pp. 3982–3992.
- [28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [30] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.
- [31] T. Mattis, T. Rausch, and M. Rinard, "RTPTorrent: An open-source dataset for evaluating regression test prioritization," in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. ACM, 2020, pp. 558–562.
- [32] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [33] B. Efron and R. J. Tibshirani, *An introduction to the bootstrap*. CRC Press, 1994.
- [34] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. ACM, 2011, pp. 1–10.

**Acauan Cardoso Ribeiro** is a researcher at the Institute of Computing (IComp), Federal University of Amazonas (UFAM), Brazil. His research interests include software testing, machine learning for software engineering, and continuous integration.

**Eduardo Luzeiro Feitosa** is a professor at the Institute of Computing (IComp), Federal University of Amazonas (UFAM), Brazil.

**Andre Luiz da Costa Carvalho** is a professor at the Institute of Computing (IComp), Federal University of Amazonas (UFAM), Brazil.

**Eulanda Miranda dos Santos** is a professor at the Institute of Computing (IComp), Federal University of Amazonas (UFAM), Brazil.

**Bruno Freitas Gadelha** is a professor at the Institute of Computing (IComp), Federal University of Amazonas (UFAM), Brazil.

**Yan Rodrigo Da Silva Soares** is a researcher at the Institute of Computing (IComp), Federal University of Amazonas (UFAM), Brazil.

**Jose Nascimento** is at Motorola Mobility LLC, Manaus, AM, Brazil.